Practical:11	Write a program for Floyd-Warshal algorithm.
__/___/2026	

Objective

- To implement the **Floyd-Warshall algorithm** to find the All-Pairs Shortest Paths in a weighted directed graph (allowing negative edge weights, provided there are no negative weight cycles).
- To understand the dynamic programming technique of using intermediate vertices to iteratively improve path costs.
- To analyze the time and space complexity of the Floyd-Warshall algorithm.

Program

```
# Floyd-Warshall Algorithm for All-Pairs Shortest Path
INF = float('inf')

def floyd_warshall(graph):
    V = len(graph)
    # Initialize distance matrix with graph values
    dist = [row[:] for row in graph]

    # Dynamic Programming: k is the intermediate vertex
    for k in range(V):
        for i in range(V):
            for j in range(V):
                # If vertex k is on the shortest path from i to j, update dist[i][j]
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    # Check for negative weight cycles
    for i in range(V):
        if dist[i][i] < 0:
            raise ValueError("Graph contains a negative weight cycle!")

    return dist
```

```
# Driver Code
if __name__== "__main__":
    # Graph represented as an adjacency matrix (V = 4)
    # INF represents no direct edge between vertices
    graph = [
        [0,   3,   INF, 7],
        [8,   0,   2,   INF],
        [5,   INF, 0,   1],
        [2,   INF, INF, 0]
    ]

    V = len(graph)

    print("FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)")
    print("\nInitial Distance Matrix (Direct Edges):")
    for row in graph:
        print([r if r != INF else "INF" for r in row])

    # Compute shortest distance matrix
    shortest_dist = floyd_warshall(graph)

    print("\nFinal Shortest Distance Matrix:")
    for row in shortest_dist:
        print([r if r != INF else "INF" for r in row])
```

Output

```
FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)

Initial Distance Matrix (Direct Edges):
[0, 3, 'INF', 7]
[8, 0, 2, 'INF']
[5, 'INF', 0, 1]
[2, 'INF', 'INF', 0]

Final Shortest Distance Matrix:
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]
```

Analysis

a) Problem Analysis

The **All-Pairs Shortest Path (APSP)** problem requires finding the minimum path cost between every pair of vertices (i, j) in a directed or undirected graph. Unlike single-source shortest path algorithms like Dijkstra's or Bellman-Ford (which start from a single source node), Floyd-Warshall computes shortest paths between all pairs simultaneously.

b) Dynamic Programming Approach

Floyd-Warshall uses a **Dynamic Programming** strategy based on intermediate vertices:

- Let $dist[i][j]^{(k)}$ be the shortest distance from vertex i to vertex j considering only vertices $\{0,1, \dots, k-1\}$ as potential intermediate nodes on the path.
- At step k, we decide whether routing through vertex k yields a shorter path from i to j than the best path found so far:

$$\text{dist}[i][j] = \min(\text{dist}[i][j],\ \text{dist}[i][k] + \text{dist}[k][j])$$

c) Example Trace

For the 4-vertex graph ($V = 0,1,2,3$):

1. **Initial Matrix ($k = -1$):**
   - $D^{(0)} = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$
2. **After $k = 0$ (using vertex 0 as intermediate):**
   - Path $3 \to 0 \to 1$: $dist[3][1] = \min(\infty, dist[3][0] + dist[0][1]) = \min(\infty, 2 + 3) = 5$

- Path $3 \rightarrow 0 \rightarrow 3$: $dist[3][3] = \min(0, 2 + 7) = 0$

3. **After $k = 1$ (using vertex 1 as intermediate):**
   - Path $0 \rightarrow 1 \rightarrow 2$: $dist[0][2] = \min(\infty, 3 + 2) = 5$
4. **After $k = 2$ (using vertex 2 as intermediate):**
   - Path $1 \rightarrow 2 \rightarrow 3$: $dist[1][3] = \min(\infty, 2 + 1) = 3$
   - Path $0 \rightarrow 2 \rightarrow 3$: $dist[0][3] = \min(7, 5 + 1) = 6$
5. **After $k = 3$ (using vertex 3 as intermediate):**
   - All entries evaluated. The final matrix gives the absolute shortest distance between every pair (i, j).

d) Algorithm Steps

1. Create a distance matrix dist initialized to the graph's weight matrix. Set dist[i][i] = 0 and non-existent edges to INF.
2. Run three nested loops:
   - Outer loop k selects the candidate intermediate vertex (0 to $V - 1$).
   - Middle loop i selects the source vertex (0 to $V - 1$).
   - Inner loop j selects the destination vertex (0 to $V - 1$).
3. Update dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).
4. If any diagonal element dist[i][i] < 0, a **negative cycle** is present in the graph.

e) Time Complexity

- Three nested loops each run V times.
- The inner operation takes $O(1)$ constant time.
- **Overall Time Complexity:** $O(V^3)$ (regardless of graph density).

f) Space Complexity

- Requires a $V \times V$ matrix to store distance values.
- **Overall Space Complexity:** $O(V^2)$ auxiliary space.

Conclusion

The Floyd-Warshall algorithm was successfully implemented in Python using Dynamic Programming. It computes all-pairs shortest paths in $O(V^3)$ time and $O(V^2)$ space, making it simple to implement and optimal for dense graphs or small to medium vertex counts ($V \leq 500$).

Practical:12	Write a program for travelling salesman problem.
__/___/2026	

Objective

- To implement the **Travelling Salesman Problem (TSP)** using **Dynamic Programming with Bitmasking** (Held-Karp Algorithm).
- To understand how state compression (bitmasking) reduces time complexity compared to brute-force recursive permutation.
- To analyze the time and space complexity of the TSP algorithm.

Program

```
# Travelling Salesman Problem (TSP) using Dynamic Programming with Bitmasking
# Held-Karp Algorithm

INF = float('inf')

# Distance matrix representing cost between cities 0, 1, 2, 3
# graph[i][j] represents distance/cost to travel from city i to city j
graph = [
    [0,  10, 15, 20],
    [10, 0,  35, 25],
    [15, 35, 0,  30],
    [20, 25, 30, 0]
]

N = len(graph)
ALL_VISITED = (1 << N) - 1  # Bitmask representing all cities visited (1111 in binary = 15)

# Memoization table: stores result for (mask, current_city)
memo = {}

def solve_tsp(mask, u):
    """
```

```
    Finds minimum cost to visit all remaining cities in 'mask'
    starting from city 'u' and returning to starting city 0.
    """
    # Base Case: All cities visited, return cost from last city back to start (city 0)
    if mask == ALL_VISITED:
        return graph[u][0]

    # Return precomputed result if already calculated
    if (mask, u) in memo:
        return memo[(mask, u)]

    min_cost = INF
    # Try visiting every unvisited city v
    for v in range(N):
        if not (mask & (1 << v)):  # Check if city v is not visited
            cost = graph[u][v] + solve_tsp(mask | (1 << v), v)
            if cost < min_cost:
                min_cost = cost

    memo[(mask, u)] = min_cost
    return min_cost

def reconstruct_path():
    """Reconstructs the optimal tour path taken by the salesman."""
    mask = 1  # Start at city 0 (bitmask 0001)
    u = 0
    path = [0]

    while mask != ALL_VISITED:
        next_city = -1
```

```
        best_val = INF
        for v in range(N):
            if not (mask & (1 << v)):
                if (mask | (1 << v)) == ALL_VISITED:
                    cost = graph[u][v] + graph[v][0]
                else:
                    cost = graph[u][v] + memo[(mask | (1 << v), v)]

                if cost < best_val:
                    best_val = cost
                    next_city = v

        path.append(next_city)
        mask |= (1 << next_city)
        u = next_city

    path.append(0)  # Return to starting city 0
    return path

# Driver Code
if __name__ == "__main__":
    print("TRAVELLING SALESMAN PROBLEM (Dynamic Programming)")

    min_cost = solve_tsp(1, 0)
    path = reconstruct_path()

    print(f"Minimum Tour Cost: {min_cost}")
    print("Optimal Tour Path: " + " -> ".join(map(str, path)))
```

Output

```
TRAVELLING SALESMAN PROBLEM (Dynamic Programming)
Minimum Tour Cost: 80
Optimal Tour Path: 0 -> 1 -> 3 -> 2 -> 0
```

Analysis

a) Problem Analysis

The **Travelling Salesman Problem (TSP)** asks: *Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?*

Formally, it requires finding a **Hamiltonian Cycle** with minimum total edge weight in a weighted graph. TSP is a classic **NP-hard** problem in combinatorial optimization with application in logistics, microchip design, and DNA sequencing.

b) Dynamic Programming & Bitmasking Approach

A naive brute-force search evaluates all $(N-1)!$ possible permutations of cities, which quickly becomes computationally infeasible.

The **Held-Karp algorithm** uses Dynamic Programming and Bitmasking to avoid redundant calculations:

1. **Bitmask Representation:** An integer's binary representation represents the subset of visited cities.
    - Example ($N = 4$): mask = 5 (0101_2) means cities 0 and 2 have been visited.
2. **DP State:** $f(\text{mask}, u)$ represents the minimum cost to visit all remaining unvisited cities, starting from city u and ultimately returning to city 0.
3. **State Transition Formula:**

$$f(\text{mask}, u) = \min_{v \notin \text{mask}} (\text{graph}[u][v] + f(\text{mask} \cup \{v\}, v))$$

c) Example Analysis

Consider $N = 4$ cities with the given cost matrix:

- **Possible Valid Tours from City 0:**
    1. $0 \to 1 \to 2 \to 3 \to 0 = 10 + 35 + 30 + 20 = 95$
    2. $0 \to 1 \to 3 \to 2 \to 0 = 10 + 25 + 30 + 15 = 80$ *(Optimal)*
    3. $0 \to 2 \to 1 \to 3 \to 0 = 15 + 35 + 25 + 20 = 95$
    4. $0 \to 2 \to 3 \to 1 \to 0 = 15 + 30 + 25 + 10 = 80$ *(Optimal Reverse)*
    5. $0 \to 3 \to 1 \to 2 \to 0 = 20 + 25 + 35 + 15 = 95$
    6. $0 \to 3 \to 2 \to 1 \to 0 = 20 + 30 + 35 + 10 = 95$

Minimum Cost: 80 via path $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$.

d) Algorithm Steps

1. Set start city as city 0 with mask = 1 (0001_2).
2. Check base case: if mask == (1 << N) - 1, return graph[u][0].
3. If state (mask, u) exists in DP memo table, return the saved result.
4. Iterate through all unvisited cities v (where bit v in mask is 0).
5. Recursively compute cost by transitioning to mask | (1 << v) and current city v.
6. Store minimum result in memo table and reconstruct path once DP computation finishes.

e) Time Complexity

- Total unique DP subproblems: $O(N \cdot 2^N)$.
- Computation time per state: $O(N)$ (looping over N candidate cities).
- **Overall Time Complexity:** $O(N^2 \cdot 2^N)$.
- *Comparison:* $O(N^2 \cdot 2^N)$ is significantly faster than brute-force $O(N!)$ for moderate values of N (e.g., $N \leq 20$).

f) Space Complexity

- Memoization table size: $O(N \cdot 2^N)$ states.
- Call stack depth: $O(N)$.
- **Overall Space Complexity:** $O(N \cdot 2^N)$.

Conclusion

The Travelling Salesman Problem was successfully implemented in Python using Dynamic Programming with Bitmasking (Held-Karp Algorithm). For the given 4-city cost matrix, the minimum tour cost obtained was **80** along the route $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$. Using bitmasking reduced the search space complexity from $O(N!)$ to $O(N^2 \cdot 2^N)$.